# Which Context Surface Do LLMs Respect Most for Org-Specific Knowledge?

An Evaluation Framework Using Controlled Retrieval Variants

Srinivas Vaddi
Independent Researcher

March 2026

## Abstract

Large language models perform well on public coding benchmarks, but may need guidance on implementations that follow org-specific policies and conventions not reflected in their training data. I present a pilot evaluation framework that explores **which context surface** — training knowledge, inline hints, oracle injection, or tool-retrieved snippets — most helps models adhere to org-specific coding policies in code repair tasks. The benchmark, **Script30**, consists of 30 composition bugs across a synthetic Python codebase: each bug requires both a logic fix *and* adherence to an org-specific convention (calling the correct `orgops.*` function with prescribed arguments) that cannot be inferred from pretraining. The benchmark measures *composition execution fidelity* — whether a model can synthesise a code fix with a retrieved org convention — rather than bug discovery; the logic error is described in the task context, and the discriminating factor is retrieval and application of the org-specific requirement. A seven-point **context surface ladder** (P0–P6) systematically varies what context the model receives, from no context (P0) to autonomous retrieval with guide documentation (P6). Across two model families (claude-haiku-4-5, $n$=3; gpt-5.1-codex-mini, $n$=3) and 1,229 scored runs evaluated via Docker-isolated pytest, I observe that P0 achieves **0.0%** on the 26-bug signal set while autonomous retrieval with a focused guide document (P4) reaches **69.2%** for Haiku — a +69.2 percentage-point lift associated with retrieval. In these runs, providing more guide context (P6) did not help over focused retrieval alone (P4) for Haiku, hinting at a possible over-specification effect. Codex showed a different pattern: static snippet injection (P3=43.0%) scored higher than tool-based retrieval (P4=20.0%) across $n$=3 runs, suggesting the preferred context surface may be model-dependent. These are directional findings from a small-$n$ pilot; the sample sizes are insufficient for statistical significance claims. The benchmark, evaluation harness, and all run artefacts are released at https://github.com/vaddisrinivas/moltsnip (branch `arxiv-v1`).

## 1 Introduction

Enterprise deployments of LLM coding assistants face a gap that public benchmarks such as HumanEval (Chen et al., 2021) and SWE-bench (Jimenez et al., 2024) do not measure: adherence to *org-specific policies* — internal coding conventions, metrics emission requirements, audit logging standards, and compliance patterns that are not reflected in any public corpus.

When a model encounters a bug whose fix must also satisfy an org policy — for example, emitting a metric via the prescribed `orgops.metrics.emit` call — it cannot infer the correct convention from training data alone. The model must obtain that knowledge from some *context surface*: inline documentation, retrieved policy snippets, or agentic tool calls.

This raises a practical question for organisations deploying LLM coding assistants:

> **Which context surface should practitioners invest in?** Training-data coverage? Inline guide prompts? Automated retrieval (RAG)? Oracle injection?

I explore this question through a controlled benchmark. The contributions are:

1. **Script30**: a 30-bug composition benchmark measuring whether models can execute code fixes that require both a logic correction and an org-specific convention, where P0 (no context) is zero on signal bugs, enabling clean lift measurement.
2. **A seven-point context surface ladder** (P0–P6) that varies context systematically while holding the model, task, and judge constant.
3. **Pilot empirical observations** across two model families: for Haiku ($n$=3), autonomous retrieval (P4=69.2%) scored higher than oracle injection (P3=65.4%), though this difference is not statistically confirmed at this sample size; for Codex ($n$=3), static injection (P3=43.0%) scored higher than dynamic retrieval (P4=20.0%).
4. **A reproducible harness** with provider-parity controls, Docker-isolated pytest ground-truth scoring, per-run retrieval tracing that links snippet coverage to task outcome, and published run artefacts.

## 2 Related Work

**Code generation benchmarks.** HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), and SWE-bench (Jimenez et al., 2024) measure model capability on public tasks. This work differs in that correctness *requires* adherence to org-specific conventions not reflected in training data, which reduces the possibility of training-data leakage as an explanation for success.

**Retrieval-augmented code generation.** Prior work has shown that retrieval improves code generation (Lewis et al., 2020), including code-specific evaluations such as CodeRAG-Bench (Wang et al., 2024) and retrieval-policy studies for code generation (Gu et al., 2025). Documentation-grounded code generation has also been studied for API usage (Chen et al., 2025). This setting differs by requiring org-specific policy conventions and by isolating retrieval surface choice (injected memory vs autonomous tools vs guide documents).

**Context retrieval and guide surfaces for coding agents.** Recent work benchmarks context retrieval behavior in coding agents (Li et al., 2026a) and studies repository-level guide files such as `AGENTS.md` (Gloaguen et al., 2026). Agent-skill benchmarks similarly evaluate how instruction packaging affects agent outcomes (Li et al., 2026b). Script30 extends this line by providing a controlled P0–P6 ladder that directly compares `SKILL.md`-style guidance, `AGENTS.md`-style guidance, and their combination under fixed tasks and tools.

**Tool-augmented LLMs.** Tool use has been studied in reasoning (Schick et al., 2023), web tasks (Yao et al., 2023), and API-centric benchmarks (Li et al., 2023), but less so in the specific setting of org-specific policy discovery during code repair.

**Agentic coding.** SWE-agent (Yang et al., 2024), Aider, and similar systems demonstrate multi-turn code repair. Knowledge-layered approaches to program repair have also shown gains from structured context injection (Ehsani et al., 2025). This work complements that line by asking not *can* agents repair bugs but *which information channel* enables correct adherence to org-specific conventions.

## 3 The Script30 Benchmark

### 3.1 Codebase

Script30 is built on a synthetic Python microservice codebase (`usecases/script30`) comprising approximately 2,000 lines of application source across six modules:

The `orgops/` module is the *information asymmetry surface*: it encodes org-specific policies (when to emit metrics, how to log audit events, what compliance decisions to record) through seven functions under the `orgops.*` namespace. These conventions are not intentionally present in any public corpus; empirically, P0 achieves 0.0% on signal bugs across all Haiku runs, consistent with models not having memorised them. Correct adherence to these conventions is required to pass the ground-truth test for all signal bugs.

### 3.2 Bug Construction

Each of the 30 bugs is a **composition bug**: it requires *two* independent fixes to pass the ground-truth test:

1. **Logic fix** — a deterministic code repair (e.g., fix lexicographic version comparison, invert a comparison operator, correct a modulo operand).
2. **Policy-compliance fix** — a call to the correct `orgops.*` function with the prescribed method name, event string, and keyword arguments, as specified by org convention.

The logic fix is solvable from training data alone; the policy-compliance fix is not. This composition structure ensures that P0 cannot pass signal bugs (the model fails on the org-convention component) while still requiring genuine reasoning for the logic component.

Bugs span five categories with six bugs each:

**Validity gating.** For each bug I verified: (a) P0 baseline is zero on signal bugs (the model cannot infer the org convention); (b) P3 oracle (correct snippets injected) is non-zero (the fix is achievable when the convention is known); (c) the required snippet is retrievable from the VoltSnip corpus.

**Bug classification.** Four bugs (BUG41, BUG59, BUG62, BUG68) show stochastic P0 passes across available Haiku runs (P0 pass rates of 25–67% when detected) and are classified as *noisy*. BUG41 is a known distractor: the source file already contains the correct exponential backoff, cap, and metric emission, so all variants pass trivially; it is retained in the benchmark for completeness but excluded from all signal claims. The remaining three noisy bugs (BUG59, BUG62, BUG68) exhibit genuine stochastic P0 passes; BUG59 exhibited a P0 pass in a separate pilot run and is conservatively retained in the noisy set. These are retained in the all-bugs analysis (Appendix B) but excluded from signal analysis. The final **signal set** comprises **26 bugs**.

### 3.3 Snippet Corpus

VoltSnip serves a corpus of **35 snippets** for Script30:

- **5 guiding-principle snippets** (one per bug category), providing general org-level patterns and policy conventions for the category.
- **30 scenario-context snippets** (one per bug), containing the specific `orgops.*` convention, expected arguments, and a code example showing correct usage.

Each task declares exactly two required snippets: the category guiding principle and the bug-specific scenario context. During dynamic retrieval (P4–P6), the model may retrieve up to 4 snippets per search query via the VoltSnip semantic search API.

Snippet code fields are truncated at 1,200 characters at retrieval time (§9.4).

Table 1: Script30 codebase modules.

| Module | Domain |
|---|---|
| `cache/` | Write-behind cache, TTL management, LFU eviction, stampede protection |
| `data/` | Migration runner, query scheduler, replication monitor |
| `logops/` | Log redaction, correlation ID propagation, sanitisation |
| `net/` | DNS cache, retry policy, TLS validation, connection pooling |
| `errors/` | Circuit breaker, dead-letter queue, error contracts, timeout escalation |
| `orgops/` | Org policy layer: metrics, auditing, alerts, compliance, health, tracing, rate limiting |

Table 2: Bug categories in Script30.

| Category | Example bugs | Example orgops convention |
|---|---|---|
| http_resilience | Exponential backoff, circuit half-open probe | `metrics.emit`, `health.report_degradation` |
| db_patterns | Version ordering, join cost selectivity | `auditing.write_event`, `metrics.emit` |
| concurrency_cache | TTL jitter sign, LFU decay, hash ring modulus | `metrics.emit`, `health.report_degradation` |
| error_contracts | Status classification, DLQ capacity, retry budget | `metrics.emit`, `alerts.notify` |
| logging_privacy | Email regex, correlation ID, timestamp UTC | `compliance.record_decision`, `tracing.annotate_span` |

# 4 The Context Surface Ladder

The framework defines seven **hypothesis variants** (P0–P6) that vary exactly one dimension of context at a time:

**P0** is the pure baseline: the model sees only the bug description, the target file contents, and a generic repair instruction. No retrieval tools, no guide documents, no snippet keys.

**P1** adds explicit instruction framing (stating acceptance criteria and expected output format) without providing any org-specific knowledge.

**P2** makes VoltSnip retrieval tools available but provides no guide document explaining how or when to search — the model must decide autonomously.

**P3** is the oracle condition: the exact required snippets are pre-fetched and injected into the system prompt. P3 provides an upper-bound estimate given correct context, though it is not a true ceiling because the model must still interpret and apply the injected knowledge.

**P4–P6** are the *autonomous retrieval* variants: the model decides whether and what to retrieve via VoltSnip MCP tools. Guide documents (SKILL.md and/or AGENTS.md) provide search strategy instructions — e.g., "search VoltSnip for relevant patterns before generating code" — without disclosing specific snippet keys.

The formal variant specification is in Appendix A.

# 5 Evaluation Harness

## 5.1 Provider Parity

I evaluated two model families via their respective CLI providers: **claude-haiku-4-5** (via Claude Code CLI) and **gpt-5.1-codex-mini** (via Codex CLI). Both are configured for maximal parity:

**Tool surface.** Both providers expose only VoltSnip MCP tools (Anthropic, 2024) for retrieval: `search_memory` (semantic search) and `get_snippet_by_canonical_key` (exact key lookup), plus legacy aliases. Filesystem tools are explicitly excluded from all variants to prevent org-policy discovery via source browsing. Claude Code blocks these via `--disallowedTools`; Codex excludes them via `include_fs_tools=False` in the HarnessMCPServer configuration.

**Working directory isolation.** Both providers run with `cwd` set to an isolated per-run temporary directory, not the repository root. This prevents the model from accessing sidecar files or source code through implicit filesystem access. Claude Code additionally blocks `Bash`, `Edit`, `Write`, and all other non-MCP tools. Codex runs with `--sandbox read-only` and wipes globally-configured MCP servers for isolation.

**Tool-call budget.** Both providers are limited to `max_tool_roundtrips=4` for tool-enabled variants (P2, P4–P6).

**Residual asymmetry.** Codex retains a read-only shell surface that Claude Code does not. Both are prevented from reaching `orgops/` source at the policy level

Table 3: Context surface ladder: seven hypothesis variants (P0–P6).

| Variant | Context surface | Tools | Retrieval | Guide docs |
|---|---|---|---|---|
| P0 | None | × | None | — |
| P1 | Explicit instruction tone | × | None | — |
| P2 | Tools only, zero guidance | ✓ | Agent decides | — |
| P3 | Oracle injection | × | Injected (exact) | — |
| P4 | Focused guide + tools | ✓ | Agent decides | SKILL.md |
| P5 | Alt. guide + tools | ✓ | Agent decides | AGENTS.md |
| P6 | Both guides + tools | ✓ | Agent decides | Both |

(read-only sandbox + cwd=tmpdir for Codex; Bash blocked + cwd=tmpdir for Claude Code). In practice, P0 scores are 0.0% on all signal bugs for both providers, consistent with the shell surface not providing meaningful advantage for org-policy discovery.

Table 4: Models evaluated and run counts.

| Provider | Model | $n$ | Scored |
|---|---|---|---|
| Claude Code | claude-haiku-4-5 | 3 | 630 |
| Codex | gpt-5.1-codex-mini | 3 | 599 |

### 5.2 Scoring

**Primary metric: pytest pass rate (ground truth).** Each generated patch is applied to an isolated Docker overlay of the target repository and the task's designated pytest test is executed. The Docker container runs with `--network none` (no internet access) and a 300-second timeout. A run *passes* if and only if pytest exits with code 0.

Each of the 30 tasks has a dedicated pytest test file (`test_bug{N}.py`) that verifies both requirements: (1) the logic fix produces correct behaviour, and (2) the correct `orgops.*` convention is followed with the prescribed function and arguments (verified via `unittest.mock.patch` interception).

**Secondary metric: LLM judge score.** An LLM judge (`openai:gpt-5.2`) evaluates each run using a structured oracle with constraint checks, success indicators, and failure modes. This follows the growing use of LLM-as-judge methodology in coding evaluation (Li et al., 2024). The judge score inflates pass rates by approximately 15–25 percentage points relative to pytest and is reported for reference but not used for primary claims.

### 5.3 Validity Gating

Several harness invariants are enforced:

- **Empty-output gate**: runs returning `status=ok` with no generated code are demoted to `status=error`.
- **Stream error detection**: Claude Code `is_error=true` stream events raise a provider error rather than producing a zero-score artefact.
- **Resume integrity**: only `status=ok` runs are cached; error runs are retried on resume.

### 5.4 Retrieval Tracing

Each task YAML specifies `required_snippet_keys`: the canonical VoltSnip keys needed to produce a correct fix (typically two keys per task: one for the logic pattern and one for the org-policy convention). The harness records per-run retrieval behaviour in the CSV artefacts:

- `required_snippet_hit_keys`: required keys that were retrieved (i.e., appeared in VoltSnip tool-call responses during the run).
- `required_snippet_missing_keys`: required keys that were *not* retrieved.
- `required_snippet_coverage`: fraction of required keys retrieved (recall per run; denominator = number of required keys for that task).
- `retrieved_snippet_keys`: all snippet keys returned by VoltSnip during the run (used to compute precision = required retrieved / total retrieved).

**Full coverage** is defined as `required_snippet_coverage = 1.0` — all required snippets were retrieved at least once. For P3 (oracle injection), snippets are injected directly rather than retrieved; the harness marks these as fully covered by construction. Post-hoc analysis script `scripts/analyze_retrieval_quality.py` aggregates these fields across all batch runs to compute the per-variant statistics reported in §8.2 and §8.3.

## 6 Experimental Setup

**Models evaluated.**

Haiku was run three times across the full 30-bug × 7-variant matrix (630 cells, 0 errors). Codex was run three times; some cells resulted in provider errors that were excluded from scoring (Run 1: 30 errors out of 210 cells; Run 2: 0 errors; Run 3: 1 error out of 210 cells). The total of 1,229 scored runs (630 Haiku + 599 Codex) excludes error-status cells.

**Scoring method.** All primary results use Docker-isolated pytest as ground truth. Runs are scored independently — no information flows between runs.

**Snippet retrieval.** VoltSnip serves 35 snippets for Script30 via a hosted MCP server. Snippet code fields are truncated to a maximum of 1,200 characters at retrieval time. This truncation is uniform across all variants that use snippets (P3–P6).

# 7 Results

## 7.1 Observed Surface Ordering

*Note: Haiku results are reported on the 26-bug signal set (excluding 4 noisy bugs where P0 passes stochastically). Codex results are reported on all 30 bugs with error-status cells excluded, as the Codex signal set has not been validated with the same noisy-bug analysis. All-bugs results for Haiku are in Appendix B.*

## 7.2 Causal Attribution

To understand whether P4 performance is associated with VoltSnip retrieval rather than filesystem browsing or training-data pattern matching, I checked four conditions:

1. **P0 = 0.0%** on all 26 signal bugs — the model cannot infer the org convention from training data alone. ✓
2. `voltsnip_tool_call_count` **> 0** for passing P4 runs — VoltSnip was actually called. ✓
3. `native_tool_call_count` **= 0** for P4 runs — no filesystem tools were available or used. ✓
4. **P4 drops to ≈0% when VoltSnip returns no snippets** — tested via an ablation run with all snippets hidden (see below). ✓

**Empty-store ablation.** To close the attribution chain, I ran one Haiku P4 evaluation ($n$=1, 26 signal bugs) with all VoltSnip snippets hidden via `is_hidden=true` in the snippet store. The model retained the SKILL.md guide and VoltSnip tools; tool calls succeeded but returned zero results. The ablation pass rate was **0.0%** (0/26) — compared to 69.2% for the standard P4 condition — confirming that the SKILL.md guide framing alone, without retrievable snippet content, does not account for the observed lift.

Conditions 1–4 together support a retrieval-attribution interpretation: the lift from P0 to P4 comes from VoltSnip retrieval content, not from guide framing or filesystem browsing.

## 7.3 Possible Over-Specification Effect (P6 < P4)

For Haiku, P6 (both guide documents + tools) scored 60.3%, lower than P4 (SKILL.md only + tools) at 69.2% — an 8.9 percentage-point gap. At $n$=3 with P4's inter-run range of 26.9 pp (Table 8), this gap is not statistically confirmed overall, but retrieval quality analysis points to a plausible mechanism.

Running `analyze_retrieval_quality.py` on the batch_1 artefacts shows that P6 achieved *full coverage* (both required snippets retrieved) in only 22.2% of runs, compared to 36.1% for P4. More strikingly, when P6 *did* retrieve both required snippets, it passed 100.0% of the time (vs 92.3% for P4 in the same condition). This suggests the P6 deficit is a *retrieval failure* — having two guide documents appears to reduce the model's ability to issue effective search queries — rather than a failure to apply retrieved knowledge once available. This is consistent with findings that models under-utilise relevant information when context is long or competing (Liu et al., 2023), though here the mechanism appears to be degraded retrieval querying rather than failure to read available context.

## 7.4 Cross-Model Divergence

The surface hierarchy appears **not consistent** across model families, though the comparison is imperfect: Haiku results are on 26 signal bugs while Codex results are on all 30 bugs (including the 4 noisy bugs), and both have $n$=3 runs:

**Observation**: Haiku showed P4 > P3 in aggregate (though Run 1 reversed this: P3=61.5% vs P4=57.7%); Codex showed P3 > P6 > P5 > P4 (static injection scored higher than dynamic retrieval). For Codex, the highest-scoring variant (P3=43.0%) uses no tools at all.

This divergence may reflect differences in tool-use proficiency between model families, but is hard to interpret given the different bug sets. Codex P3 per-run values were: Run 1: 46.2%, Run 2: 36.7%, Run 3: 46.7% — relatively stable across runs (range 10 pp). By contrast, Codex P4 showed much greater instability: Run 1: 46.2%, Run 2: 6.7%, Run 3: 10.0% — a 40 pp swing between Run 1 and Runs 2–3. Run 3 (P4=10.0%) is consistent with Run 2 rather than Run 1, which strengthens the observation that Codex struggles with VoltSnip tool-based retrieval in most runs. Retrieval quality analysis (`analyze_retrieval_quality.py`) shows Codex P4 achieved full-coverage retrieval in only 3.6% of runs across the first two Codex runs, compared to 36.1% for Haiku P4; the third run is consistent with this pattern given its P4 pass rate of 10.0%. This near-zero retrieval recall in Codex P4 — with two of three runs showing ≤10% pass rates — suggests that Codex was largely unable to issue effective VoltSnip queries in most runs, rather than reflecting a stable model preference for static injection.

## 7.5 Per-Run Variance (Haiku)

Haiku's three runs show moderate inter-run variance:

P4 shows the widest range (26.9 pp), consistent with the stochastic nature of autonomous retrieval — the model's retrieval strategy varies across runs. P0 is perfectly stable at 0.0% across all three runs, consistent with the zero-leakage design.

Table 5: Pytest pass rates by variant — Haiku (claude-haiku-4-5, $n$=3, 26 signal bugs).

| Variant | Pass rate | Lift vs P0 | Description |
|---|---|---|---|
| P0 | **0.0%** (0/78) | — | Baseline: training data only |
| P1 | 2.6% (2/78) | +2.6 pp | Explicit criteria hint |
| P2 | 39.7% (31/78) | +39.7 pp | Tools, no guide |
| P3 | 65.4% (51/78) | +65.4 pp | Oracle injection |
| P4 | **69.2%** (54/78) | +69.2 pp | SKILL.md + autonomous retrieval |
| P5 | 67.9% (53/78) | +67.9 pp | AGENTS.md + autonomous retrieval |
| P6 | 60.3% (47/78) | +60.3 pp | Both guides + autonomous retrieval |

Table 6: Pytest pass rates by variant — Codex (gpt-5.1-codex-mini, $n$=3, all 30 bugs, errors excluded).

| Variant | Pass rate | Lift vs P0 | Description |
|---|---|---|---|
| P0 | 4.7% (4/86) | — | Baseline |
| P1 | 5.9% (5/85) | +1.2 pp | Explicit criteria hint |
| P2 | 26.7% (23/86) | +22.0 pp | Tools, no guide |
| P3 | **43.0%** (37/86) | +38.3 pp | Oracle injection |
| P4 | 20.0% (17/85) | +15.3 pp | SKILL.md + autonomous retrieval |
| P5 | 24.4% (21/86) | +19.7 pp | AGENTS.md + autonomous retrieval |
| P6 | 30.6% (26/85) | +25.9 pp | Both guides + autonomous retrieval |

# 8 Analysis and Discussion

## 8.1 P4 vs P3: A Possible Retrieval-Cueing Effect

P3 injects the exact correct snippets as fixed context, while P4 requires the model to retrieve them autonomously. The aggregate result that P4 (69.2%) $\geq$ P3 (65.4%) for Haiku is interesting but not definitive: Run 1 showed P3 > P4 (61.5% vs 57.7%), so the pattern does not hold in every run.

One speculative explanation is that the act of retrieval itself cues the model to attend to and apply the retrieved knowledge, rather than treating injected context as background that can be overlooked. An alternative explanation is that P4's SKILL.md guide document provides additional value beyond retrieval cueing — it frames *how* to apply retrieved knowledge, not just *what* to retrieve. Retrieval quality analysis supports the second explanation: P4 passing runs ($N$=54) used a mean of 1.4 tool calls vs 0.7 for failing runs ($N$=24), and passing runs achieved full coverage (both required snippets retrieved) at a much higher rate. P2 (tools, no guide) also shows a stark gap: when P2 retrieved both required snippets it passed 89.5% of the time, but full coverage only occurred in 26.4% of P2 runs vs 36.1% for P4. This suggests the SKILL.md guide raises retrieval success rate by directing the model to search more effectively, not just by providing framing context.

## 8.2 Guide Document Effects

SKILL.md (P4) scored slightly higher than AGENTS.md (P5) for Haiku, and both scored higher than their combination (P6). This is consistent with the idea that guide documents are useful for directing attention to retrieval, but stacking guides may introduce noise rather than signal. This is consistent with SkillsBench's finding that focused Skills with 2–3 modules outperform comprehensive documentation (Li et al., 2026b), though the margins here are small and based on $n$=3 runs.

## 8.3 Codex Divergence

Codex's preference for static injection (P3) over dynamic retrieval (P4/P5) may reflect several factors:

1. **Tool-use proficiency**: Codex may issue fewer or less effective VoltSnip search queries than Haiku.
2. **MCP integration maturity**: The Codex HarnessMCPServer uses HTTP URL-based transport; protocol edge cases during the study required additional compliance work.
3. **Inter-run variance**: Codex Runs 2 and 3 both showed low pass rates on tool-enabled variants (P4: 6.7% and 10.0% respectively, vs Run 1: 46.2%), with Run 3 corroborating rather than explaining the Run 2 pattern. The persistence across two additional runs suggests this is not a transient infrastructure effect, but the mechanism remains unclear.

## 8.4 Tentative Implications for Practitioners

If these pilot observations hold at larger $n$, the observed Haiku hierarchy (P4 > P5 > P3 > P6 > P2 > P1 $\approx$ P0) would suggest:

- **Retrieval tooling may matter more than prompt engineering.** The gap P4 − P1 = +66.6 pp for Haiku in these runs.
- **One focused guide document may be enough.** P4 scored higher than P6 by 8.9 pp, though this is not statistically confirmed.

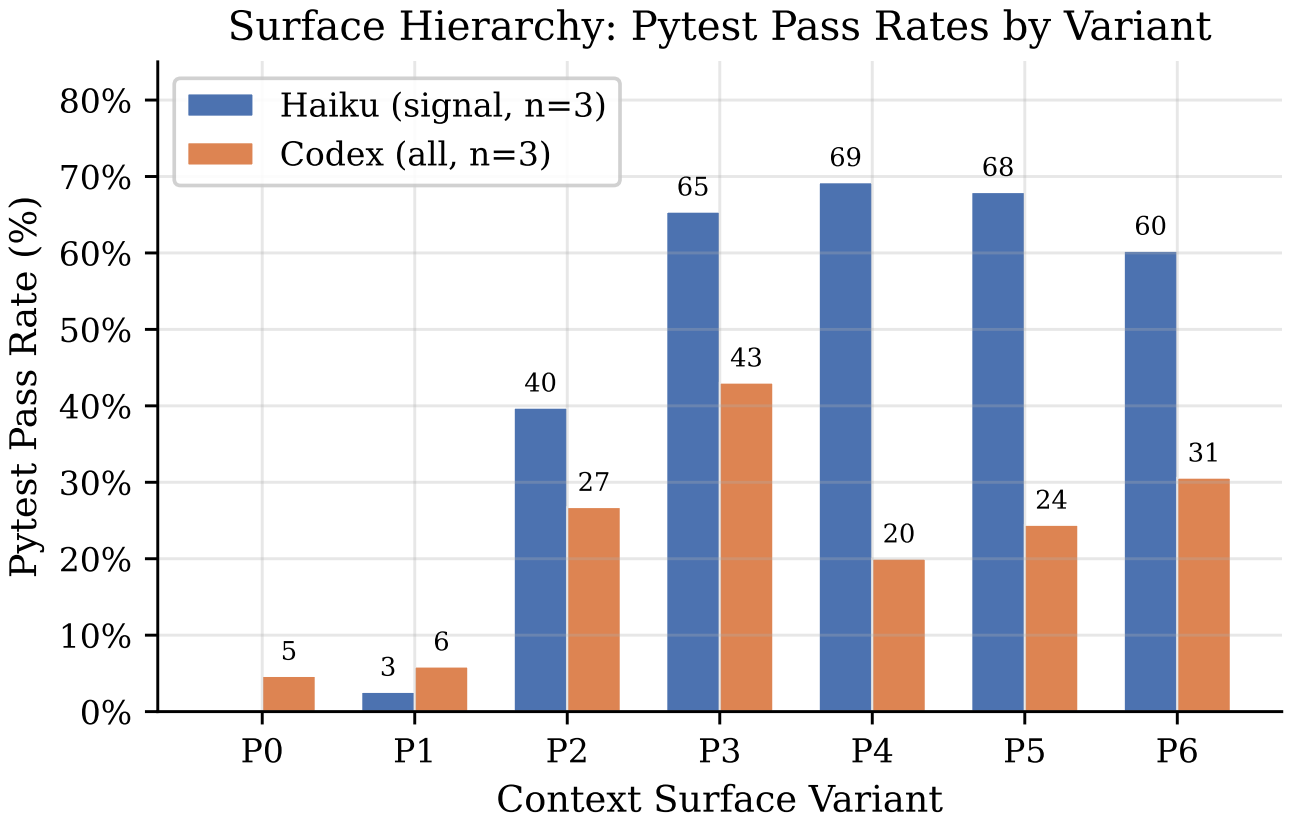


Figure 1: Pytest pass rates by context surface variant for Haiku (26 signal bugs, $n$=3) and Codex (all 30 bugs, $n$=3, errors excluded). P4 (focused guide + autonomous retrieval) showed the highest mean rate for Haiku in these runs; Codex peaked at P3 (oracle injection). Note: Haiku and Codex are on different bug sets and are not directly comparable.

- **Oracle injection may not be necessary**: autonomous retrieval (P4=69.2%) scored comparably to oracle injection (P3=65.4%), which matters because oracle injection requires upfront human curation of per-task relevant snippets — expensive at scale.
- **For Codex-family models**: the data tentatively suggests static injection may work better than tool-based retrieval. With $n$=3 runs, P3 was stable (range 10 pp) while P4 was highly variable (range 40 pp, with two of three runs at $\leq$10%), pointing to a tool-use issue rather than a model-level preference.

# 9 Limitations and Threats to Validity

## 9.1 Benchmark Construction

The benchmark was constructed by the author with knowledge of both bugs and required snippets. To mitigate bias: (a) all bugs were validated end-to-end with P0=0 and P3>0 checks; (b) four noisy bugs are excluded from signal analysis with documented rationale; (c) ground-truth scoring uses Docker-isolated pytest, not the author's judgement. One bug (BUG41) is a known distractor whose source was inadvertently fixed prior to evaluation; it passes at P0 for all models and is excluded from signal analysis. There is no third-party validation or inter-annotator agreement on the bug set; the benchmark should be treated as a single-author construction with the corresponding caveats.

The benchmark measures *composition execution* — whether a model can synthesise a code fix with a retrieved org convention — rather than bug *discovery*. The logic error is described in inline source comments (e.g., `# BUG_42: should be < not >`), so models do not need to locate or diagnose the bug; the discriminating factor is whether the model retrieves and correctly applies the org-specific convention. This design is intentional: it isolates the retrieval-attribution question from the bug-finding question. However, it means the benchmark does not evaluate debugging ability, and the P0 baseline reflects failure to produce the `orgops.*` call rather than failure to identify the bug.

## 9.2 Snippet Fidelity

The 30 bug-specific scenario snippets provide high-fidelity guidance: each contains both the required logic pattern and the exact `orgops.*` API call with prescribed arguments. The 5 category-level guiding-principle snippets similarly enumerate the correct conventions for each bug in that category. This means that a model which successfully retrieves the relevant snippet

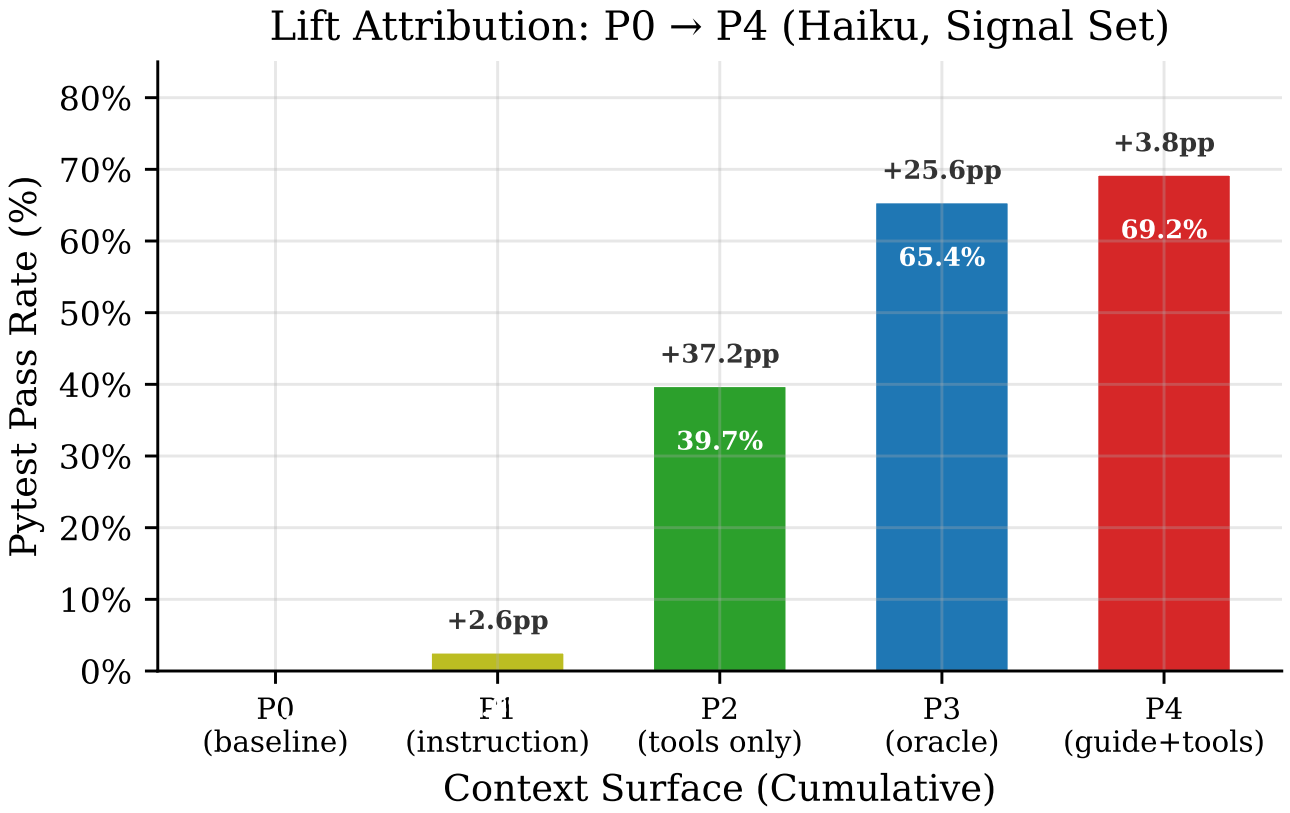


Figure 2: Cumulative lift from P0 (baseline) to P4 (highest-scoring variant) for Haiku on the 26-bug signal set. Each bar shows the absolute pass rate; annotations show incremental lift from the previous variant.

Table 7: Cross-model divergence on selected variants. Haiku: 26 signal bugs, $n$=3 runs. Codex: all 30 bugs, $n$=3 runs. The Δ column is approximate because the bug sets differ; direct comparison should be treated with caution.

| Variant | Haiku | Codex | Δ (H−C) | Bug set |
|---|---|---|---|---|
| P0 | 0.0% | 4.7% | −4.7 pp | ≈ |
| P3 (oracle) | 65.4% | **43.0%** | +22.4 pp | ≈ |
| P4 (SKILL.md + tools) | **69.2%** | 20.0% | +49.2 pp | ≈ |
| P5 (AGENTS.md + tools) | 67.9% | 24.4% | +43.5 pp | ≈ |
| P6 (both + tools) | 60.3% | 30.6% | +29.7 pp | ≈ |

≈ = not directly comparable (different bug sets).

receives near-complete specification of the required fix, reducing the task to executing a provided spec rather than inferring the convention from abstract principles.

In a production setting, org knowledge bases would contain more abstract guidance (e.g., "circuit breaker state transitions require health reporting") rather than per-bug solutions. The high snippet fidelity likely inflates absolute pass rates for snippet-consuming variants (P3–P6) relative to what would be observed with more realistic, abstract snippets. However, it does not affect the *relative* comparison between P3 (oracle injection) and P4 (autonomous retrieval), since both consume the same snippet content — the difference is in the delivery mechanism. The P0 = 0.0% baseline is also unaffected, since P0 receives no snippets.

### 9.3 Uniform Difficulty and Uncalibrated Baselines

All 30 tasks are labelled `difficulty: hard` with a uniform `trap_baseline` of 0.15, despite logic fixes ranging from trivial (single-character operator flip, e.g. BUG42: `>` to `<`) to moderately complex (implementing recursive traversal with base64 decoding, e.g. BUG70). The trap baselines were not empirically calibrated from P0 data; they are placeholder values. Future work could introduce difficulty tiers and per-task baselines derived from observed P0 pass rates.

### 9.4 Snippet Truncation

Snippet code fields are truncated to 1,200 characters at retrieval time. Of the 30 bug-specific scenario snippets, 12 exceed this limit and lose their trailing content — typically a reinforcement paragraph stating that both

Table 8: Per-run pass rates for Haiku (26 signal bugs).

| Variant | Run 1 | Run 2 | Run 3 | Mean | Range |
|---|---|---|---|---|---|
| P0 | 0.0% | 0.0% | 0.0% | 0.0% | 0 pp |
| P2 | 30.8% | 42.3% | 46.2% | 39.7% | 15.4 pp |
| P3 | 61.5% | 69.2% | 65.4% | 65.4% | 7.7 pp |
| P4 | 57.7% | 84.6% | 65.4% | 69.2% | 26.9 pp |
| P6 | 65.4% | 61.5% | 53.8% | 60.3% | 11.6 pp |

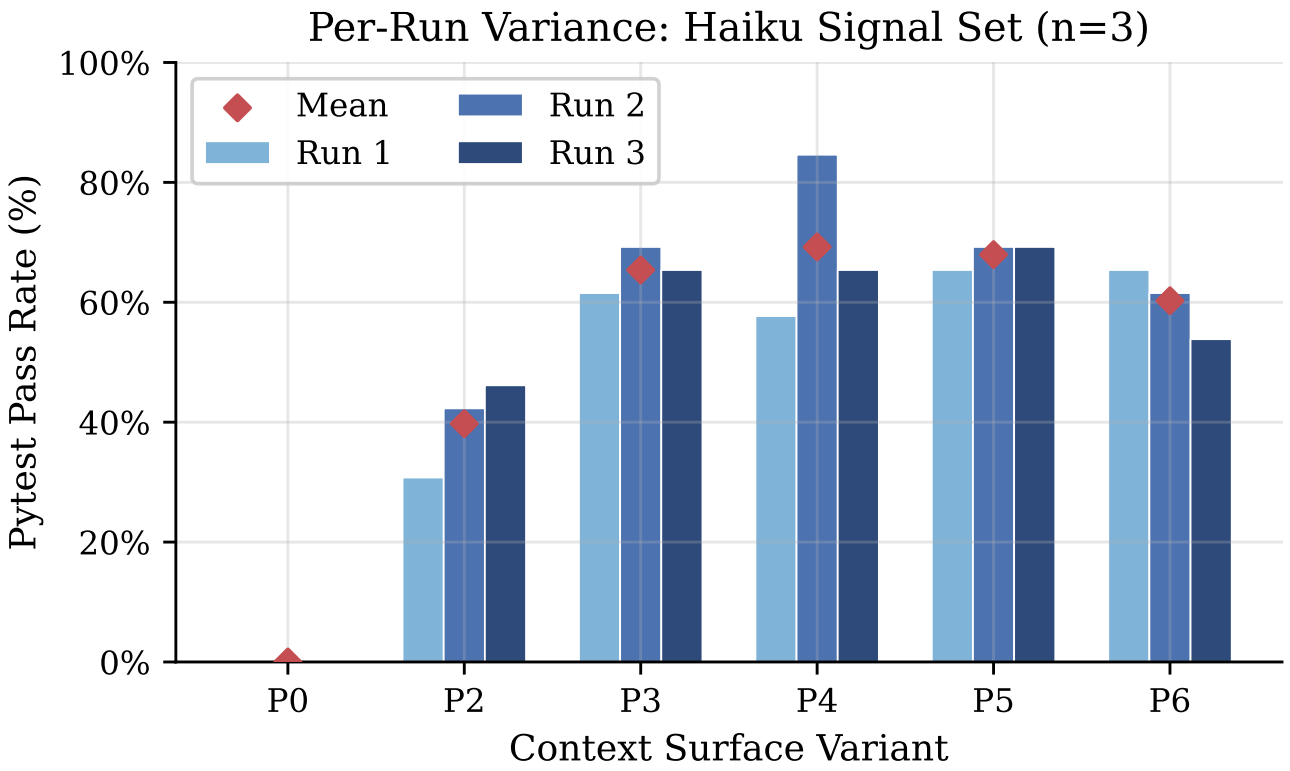


Figure 3: Per-run pass rates for Haiku across three independent runs (26 signal bugs). P4 shows the widest inter-run variance (26.9 pp range), reflecting stochastic retrieval strategy. P0 is stable at 0.0% across all runs.

fixes are required. The `orgops.*` convention details are preserved in the non-truncated portion for all 12 affected snippets. This truncation is uniform across all snippet-consuming variants (P3–P6) and does not create a differential bias between variants. Raising the limit to 2,000 characters would eliminate all truncation; I report results with the 1,200-character limit as-is.

### 9.5 Scoring Validity

The primary metric (pytest) is deterministic and objective: the test either passes or fails. Each test verifies both the logic fix and the `orgops.*` convention via mock interception. The tests are coupled to specific keyword argument names prescribed by org convention (e.g., `utilization_pct`, `column_type`), which means a model that uses a different keyword name would fail. This coupling is by design — the policy snippets specify exact conventions, and the benchmark tests whether the model follows retrieved guidance precisely.

The LLM judge (secondary metric) inflates pass rates by 15–25 pp relative to pytest and is not used for primary claims.

### 9.6 Scale

Both Haiku and Codex were evaluated with $n$=3 runs per cell. Even at $n$=3, per-variant estimates remain noisy: Codex P4 showed a 40 pp swing between Run 1 (46.2%) and Runs 2–3 ($\leq$10%), making the per-variant mean sensitive to individual run outcomes. Haiku's $n$=3 provides moderate consistency; the key observations (P0=0.0%, P4 as highest-scoring variant) held across all three runs, though Run 1 showed P3 > P4.

### 9.7 Corpus Size

The VoltSnip snippet corpus contains 35 snippets. A production knowledge base would be orders of magnitude larger, increasing retrieval difficulty. Corpus size is fixed across variants (all variants share the same corpus) so relative comparisons are valid, but absolute retrieval difficulty is likely underestimated relative to production settings.

### 9.8 Single Codebase

All bugs are in the Script30 codebase, a single synthetic repository. Generalisability to real codebases with different naming conventions, code density, and module structures is not established.

### 9.9 Residual Provider Asymmetry

Codex retains a read-only shell surface while Claude Code's shell is fully blocked. Both are prevented from accessing `orgops/` source through working-directory isolation and tool restrictions. P0=0.0% on signal bugs for both providers is consistent with this asymmetry not affecting the primary finding, but cross-provider comparisons should be interpreted with this caveat.

## 10 Future Work

Several extensions would strengthen and generalise these observations:

1. **Additional model families.** Evaluating claude-sonnet-4-6, claude-opus-4-6, gpt-5.2-codex, and open-weight models would test whether the surface hierarchy generalises.

2. **Additional Codex runs.** With $n$=3, Codex P3 is moderately stable but P4 remains highly variable (40 pp range). Increasing to $n$=5 or higher and inspecting per-run tool traces would help distinguish infrastructure instability from a stable tool-use limitation.

3. **Abstract snippet rewrites.** Rewriting scenario snippets to provide principle-level guidance (e.g., "state transitions require health reporting") rather than per-bug specifications would test whether models can *infer* the correct convention from abstract guidance, more closely modelling production knowledge bases.

4. **Corpus scaling.** Increasing the snippet corpus from 35 to 500+ (with distractors) would test retrieval robustness under more realistic conditions.

5. **Difficulty gradation.** Annotating tasks with difficulty tiers based on observed pass rates would enable per-difficulty analysis.

6. **Negative controls.** Adding tasks where no `orgops.*` convention applies (pure logic fixes) would test for over-application of retrieved patterns.

7. **Guide document ablation.** Testing additional guide document styles (minimal vs verbose, directive vs suggestive) would further characterise the over-specification effect.

8. **Tool-call and retrieval analysis (partially completed).** Per-variant tool-call counts and retrieval recall have been computed post-hoc via two analysis scripts in the repository; results inform §8.2 and §8.3. Extending to per-bug granularity and per-run snippet-utilisation traces would further distinguish "retrieved but not applied" from "never retrieved" failure modes.

## 11 Conclusion

This paper introduced Script30, a 30-bug composition benchmark for measuring LLM adherence to org-specific coding policies, and a seven-point context surface ladder (P0–P6) for exploring which information channel contributes to correct fixes.

The pilot results show P0 = 0.0% on signal bugs (the org convention cannot be inferred from training data) and that autonomous VoltSnip retrieval with a focused guide document (P4) reached 69.2% for Haiku ($n$=3) — a +69.2 pp lift associated with retrieval. An empty-store ablation (P4 with all snippets hidden) confirmed that the SKILL.md guide framing alone does not account for this lift. A post-hoc retrieval quality analysis identified a plausible mechanism for the P6 deficit: P6 achieved full-coverage retrieval (both required snippets retrieved) in only 22.2% of runs vs 36.1% for P4, yet passed 100% of the time when full coverage was achieved — indicating a retrieval failure, not an application failure. The possible over-specification effect (P6 < P4 for Haiku) and the cross-model divergence (Haiku favoured dynamic retrieval; Codex P3 was stable across $n$=3 runs at 37–47% while Codex P4 was unstable with two of three runs at ≤10%, suggesting a tool-use issue rather than a model-preference effect) are directional observations that need larger samples to confirm.

The main contribution is the benchmark and evaluation framework itself — the context surface ladder design is reusable across models and retrieval backends. The framework and all run artefacts are publicly released at https://github.com/vaddisrinivas/moltsnip to enable replication and extension.

## References

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish,

Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. https://arxiv.org/abs/2107.03374

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. https://arxiv.org/abs/2108.07732

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. https://arxiv.org/abs/2310.06770

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. https://arxiv.org/abs/2405.15793

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv preprint arXiv:2005.11401*, 2020. https://arxiv.org/abs/2005.11401

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. https://arxiv.org/abs/2302.04761

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023. https://arxiv.org/abs/2210.03629

Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. ContextBench: A benchmark for context retrieval in coding agents. *arXiv preprint arXiv:2602.05892*, 2026. https://arxiv.org/abs/2602.05892

Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. Evaluating `AGENTS.md`: Are repository-level context files helpful for coding agents? *arXiv preprint arXiv:2602.11988*, 2026. https://arxiv.org/abs/2602.11988

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, and Jiankai Sun. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026. https://arxiv.org/abs/2602.12670

Zora Zhiruo Wang, Akari Asai, Xinyan Velocity Yu, Frank F. Xu, Yiqing Xie, Graham Neubig, and Daniel Fried. CodeRAG-Bench: Can retrieval augment code generation? *arXiv preprint arXiv:2406.14497*, 2024. https://arxiv.org/abs/2406.14497

Wenchao Gu, Juntao Chen, Yanlin Wang, Tianyue Jiang, Xingzhe Li, Mingwei Liu, Xilin Liu, Yuchi Ma, and Zibin Zheng. What to retrieve for effective retrieval-augmented code generation? An empirical study and beyond. *arXiv preprint arXiv:2503.20589*, 2025. https://arxiv.org/abs/2503.20589

Jingyi Chen, Songqiang Chen, Jialun Cao, Jiasi Shen, and Shing-Chi Cheung. When LLMs meet API documentation: Can retrieval augmentation aid code generation? *arXiv preprint arXiv:2503.15231*, 2025. https://arxiv.org/abs/2503.15231

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. *arXiv preprint arXiv:2304.08244*, 2023. https://arxiv.org/abs/2304.08244

Ramtin Ehsani, Esteban Parra, Sonia Haiduc, and Preetha Chatterjee. Hierarchical knowledge injection for improving LLM-based program repair. *arXiv preprint arXiv:2506.24015*, 2025. https://arxiv.org/abs/2506.24015

Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. LLMs-as-judges: A comprehensive survey. *arXiv preprint arXiv:2412.05579*, 2024. https://arxiv.org/abs/2412.05579

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2023. https://arxiv.org/abs/2307.03172

Anthropic. Model Context Protocol specification. Technical report, Anthropic, 2024. https://modelcontextprotocol.io/specification/

# A Variant Ladder Formal Specification

# B All-Bugs Results (30 bugs, Haiku $n$=3)

*Note: All-bugs results include the 4 noisy bugs (BUG41, BUG59, BUG62, BUG68) where P0 passes stochastically. The P0 rate of 5.6% reflects passes on noisy bugs*

Table 9: Formal specification of the seven hypothesis variants.

| Field | P0 | P1 | P2 | P3 | P4 | P5 | P6 |
|---|---|---|---|---|---|---|---|
| `mode` | direct | direct | agent | agent | agent | agent | agent |
| `memory_enabled` | × | × | × | ✓ | × | × | × |
| `tools_enabled` | × | × | ✓ | × | ✓ | ✓ | ✓ |
| `retrieval_mode` | none | none | agent_dec. | injected | agent_dec. | agent_dec. | agent_dec. |
| `instruction_mode` | none | explicit | none | none | none | none | none |
| `context_surface` | user | user | tools_only | system | skills_md | agents_md | both_md |
| `max_tool_rtrips` | 1 | 1 | 4 | 4 | 4 | 4 | 4 |

Table 10: Pytest pass rates on all 30 bugs (Haiku, $n$=3).

| Variant | Pass rate | Lift vs P0 |
|---|---|---|
| P0 | 5.6% (5/90) | — |
| P1 | 6.7% (6/90) | +1.1 pp |
| P2 | 42.2% (38/90) | +36.6 pp |
| P3 | 65.6% (59/90) | +60.0 pp |
| P4 | 71.1% (64/90) | +65.5 pp |
| P5 | 71.1% (64/90) | +65.5 pp |
| P6 | 61.1% (55/90) | +55.5 pp |

*only. P4 and P5 are tied at 71.1% in the all-bugs set despite P4 (69.2%) > P5 (67.9%) on signal bugs; the noisy bugs slightly favour P5, erasing P4's signal-set advantage.*

# C Harness Integrity Fixes Applied During Study

Three harness bugs were discovered and corrected during the study:

1. **Stream error detection** (`claudecode.py`): Claude Code `is_error=true` stream events were not propagated as provider errors, silently producing `status=ok` runs with empty code. Fixed by adding a stream error extractor.
2. **Empty-output validity gate** (`runner.py`): runs with `status=ok` but no generated code were not demoted to error. Fixed by post-run validity check.
3. **Resume integrity** (`csv_mapper.py`): `load_completed` loaded all entries regardless of status, preventing error runs from being retried on resume. Fixed by filtering to `status=ok`.

All affected runs were re-executed after fixes were applied.

# D Artefact Paths

All run artefacts (full model outputs, tool traces, scoring results, and pytest results) are available at:

https://github.com/vaddisrinivas/moltsnip

The repository uses two branches:

- `arxiv-v1` — benchmark code, evaluation harness, paper source, and all paths below.
- `arxiv-v1` sub-directory `vsevals_runs/batch_1/` — raw evaluation run artefacts (CSV results, full model dumps, pytest logs, tool traces).

## Claim-to-Evidence Mapping

Every quantitative claim in this paper is mechanically derived from the CSV artefacts listed above via `scripts/build_paper.py`. The following table maps key claims to the run files and column fields that produce them. Running `uv run python3 scripts/build_paper.py -check` reproduces all numbers from the raw CSVs.

Table 11: Artefact locations in the repository (all paths relative to `vsevals/`).

| **Artefact** | **Path** |
|---|---|
| Suite definition | `suites/script30.yaml` |
| Task definitions (30 tasks) | `tasks/script30/BUG{41..70}.yaml` |
| Snippet corpus (35 snippets) | `snippets/script30/` |
| Usecase codebase | `usecases/script30/` |
| Pytest tests (30 tests) | `usecases/script30/tests/unit/test_bug{41..70}.py` |
| Haiku Run 1 | `.../batch_1/matrix_20260303T193221*` |
| Haiku Run 2 | `.../batch_1/matrix_20260303T201709*` |
| Haiku Run 3 | `.../batch_1/matrix_20260303T215606*` |
| Codex Run 1 | `.../batch_1/matrix_20260303T193214*` |
| Codex Run 2 | `.../batch_1/matrix_20260303T201241*` |
| Codex Run 3 | `.../batch_1/matrix_20260304T083357*` |
| Ablation (empty-store) | `.../batch_1/matrix_20260304T095907*` |
| Experiment log | `EXPERIMENT.md` |
| Evaluation harness source | `vsevals/` |
| Build script | `scripts/build_paper.py` |

Table 12: Evidence mapping: paper claims to run artefacts and CSV fields.

| **Claim** | **Evidence source** |
|---|---|
| P0 = 0.0% on 26 signal bugs (Haiku) | Haiku Runs 1–3, `matrix_results.csv`: filter `variant_id=P0`, exclude noisy bugs, count `pytest_passed=True` → 0/78. |
| P4 = 69.2% (Haiku, 26 signal) | Haiku Runs 1–3, same CSV: filter `variant_id=P4` → 54/78 passed. |
| P3 = 43.0% (Codex, all 30 bugs) | Codex Runs 1–3, same CSV: filter `variant_id=P3`, exclude `status≠ok` → 37/86 passed. |
| Codex P4 = 20.0% | Codex Runs 1–3: filter `variant_id=P4`, exclude errors → 17/85. |
| P4 passing runs: 1.4 mean tool calls | Haiku Runs 1–3, CSV column `tool_call_count`: filter P4 signal, split by `pytest_passed`. $N$=54 passing, $N$=24 failing. |
| P6 full-coverage = 22.2% vs P4 = 36.1% | `scripts/analyze_retrieval_quality.py` on batch_1: column `required_snippet_coverage=1.0`. |
| Ablation P4 (empty store) | Ablation run CSV: filter `variant_id=P4`, signal bugs only. |
| Run error counts (30/0/1) | Per-run: count rows where `pytest_passed` is empty in each Codex run CSV. |